

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 5**



Searching

Oleh:

Nuzulia Sekti Mahanania NIM. 2510817220006

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 5

Laporan Praktikum Algoritma & Struktur Data Modul 5: Searching ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nuzulia Sekti Mahanania
NIM : 2510817220006

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL	iv
Linear Search.....	1
A. Source Code.....	1
B. Output Program	2
C. Pembahasan	2
Binary Search	4
A. Source Code.....	4
B. Output Program	5
C. Pembahasan	5
TAUTAN GITHUB.....	8

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Linear Search	2
Gambar 2. Screenshot Hasil Binary Search.....	5

DAFTAR TABEL

Tabel 1. Source Code Linear Search	1
Tabel 2. Source Code Binary Search.....	4

Linear Search

A. Source Code

Tabel 1. Source Code Linear Search

1	#include "searching_algorithms.h"
2	#include <chrono>
3	
4	
5	
6	using Clock = std::chrono::high_resolution_clock;
7	
8	
9	void linear_search(const std::vector<int>& data, int target, Metrics& m) {
10	auto start = Clock::now();
11	
12	m.found_index = -1;
13	m.comparisons = 0;
14	
15	for (int i = 0; i < data.size(); ++i) {
16	m.comparisons++;
17	if (data[i] == target) {
18	m.found_index = i;
19	break;
20	}
21	}
22	
23	m.time_us = std::chrono::duration<double, std::micro>(Clock::now() - start).count();
24	}

B. Output Program

```
Windows PowerShell
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\searching> g++ -std=c++23 -O0 main.cpp data_generator.cpp searching_algorithms.cpp reporter.cpp -o search
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\searching> ./search

=====
SEARCHING ALGORITHM - ANALYSIS
=====

>> Linear Search

-----
Algorithm      Case      N      Comparisons  Found Index  Time(us)
-----
Linear Search   Best      1000    1             0             1.100
Linear Search   Average    1000    375           374           4.300
Linear Search   Worst     1000    1000          Not Found     9.900
-----
Linear Search   Best     10000    1             0             0.500
Linear Search   Average  10000   3746          3745          60.300
Linear Search   Worst   10000   10000         Not Found     95.100
-----
Linear Search   Best    100000    1             0             0.500
Linear Search   Average 100000  37455         37454         362.000
Linear Search   Worst  100000  100000        Not Found     975.200
-----
Linear Search   Best   1000000    1             0             1.400
Linear Search   Average 1000000  374541        374540        4746.100
Linear Search   Worst  1000000  1000000       Not Found    11425.200
-----
```

Gambar 1. Screenshot Hasil Linear Search

C. Pembahasan

1. Karakteristik Algoritma

Secara definisi, Linear Search adalah algoritma pencarian yang bekerja dengan menelusuri setiap elemen di dalam kumpulan data satu per satu secara berurutan. Karena yang diinginkan adalah mencari elemen dari ujung awal hingga akhir, maka pakai perulangan `for` mulai dari indeks 0 sampai batas sizenya. Untuk mencari elemen mana yang sesuai, pertama program akan membaca nilai pada array di indeks ke-*i* tersebut. Kemudian, nilai itu dicocokkan dengan nilai target menggunakan `if`. Jika nilainya sama, maka indeksnya disimpan lalu perulangan dihentikan menggunakan `break`. Jika nilainya tidak sama, maka nilai `i` akan dinaikkan (`++i`) sehingga iterasi bergeser menelusuri indeks selanjutnya. Kalau sampai perulangan berakhir elemen target tidak juga ditemukan, maka biarkan indeks pencariannya tetap bernilai -1.

Karena algoritma ini bekerja hanya membaca nilai indeks per indeks tanpa ada pengecekan urutan angka sebelum atau sesudahnya, maka algoritma tidak memerlukan data terurut. Lalu karena program hanya berputar di dalam blok `for` tanpa adanya fungsi yang memanggil dirinya sendiri, maka algoritma ini bersifat iteratif. Namun, algoritma ini sebenarnya dapat diimplementasikan secara rekursif dengan cara fungsinya

memanggil dirinya sendiri sambil memajukan nilai indeks satu per satu sebagai pengganti perulangan.

2. Kompleksitas Waktu

Jika elemen target yang dicari kebetulan langsung ditemukan pada indeks pertama ($i = 0$), perintah `break` di dalam kondisi tersebut akan langsung dieksekusi untuk menghentikan paksa perulangan saat itu juga. Karena algoritmanya hanya membutuhkan satu langkah operasi saja dan langsung keluar dari perulangan tanpa memedulikan banyak sisa datanya, maka kompleksitas waktu untuk best case adalah $O(1)$.

Sebaliknya, jika elemen target di posisi acak, berada di ujung, atau bahkan tidak ada sama sekali, perintah `break` tidak akan bisa tercapai dengan cepat. Kode perulangan `for` ini akan memaksa program untuk terus menelusuri dan mencocokkan data satu per satu secara berulang hingga i menyentuh batas akhir ukuran data (`data.size()`). Karena jumlah langkah pengecekan ini akan terus meningkat bersamaan dengan bertambahnya ukuran total data (n), maka kompleksitas waktu untuk average dan worst case adalah $O(n)$.

3. Analisis Operasi

Pada eksperimen Linear Search, jumlah comparison yang dilakukan bergantung pada letak datanya. Best Case memakan 1 kali perbandingan, Average Case berada di pertengahan (374.541 kali pada 1 juta data), dan Worst Case memeriksa penuh seluruh data (1.000.000 kali). Jika elemen ditemukan, posisinya langsung disimpan ke variabel `index_found` (seperti indeks ke-0 atau 374540), namun jika seluruh data habis diperiksa tanpa hasil, `index_found` dikembalikan bernilai -1 (Not Found). Karena penelusurannya berurutan dari indeks terawal, posisi elemen target sangat menentukan jumlah comparison, target di depan mempercepat pencarian, sedangkan target di belakang memperlama pencarian. Karena alurnya mengecek array satu per satu, maka besarnya ukuran data (n) akan memberikan pengaruh peningkatan jumlah operasi secara linear.

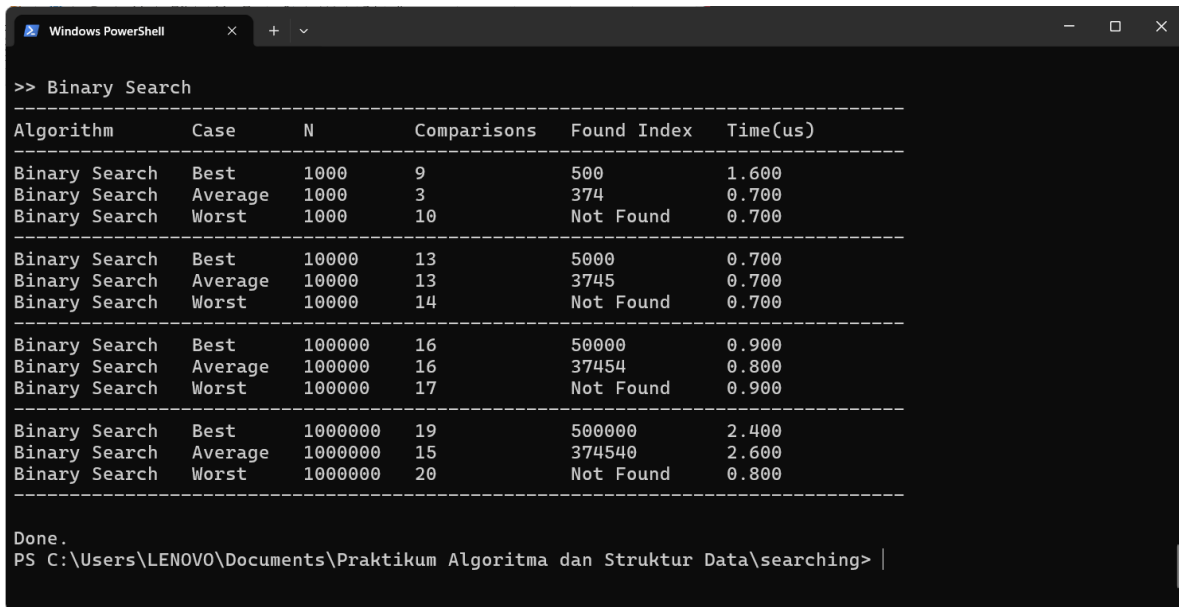
Binary Search

A. Source Code

Tabel 2. Source Code Binary Search

```
28 void binary_search(const std::vector<int>& data, int
   target, Metrics& m) {
29     auto start = Clock::now();
30
31     m.found_index = -1;
32     m.comparisons = 0;
33
34     int left = 0;
35     int right = data.size() - 1;
36
37     while (left <= right) {
38         int mid = left + (right - left) / 2;
39
40         m.comparisons++;
41         if (data[mid] == target) {
42             m.found_index = mid;
43             break;
44         } else if (data[mid] < target) {
45             left = mid + 1;
46         } else {
47             right = mid - 1;
48         }
49     }
50
51     m.time_us = std::chrono::duration<double,
   std::micro>(Clock::now() - start).count();
52 }
```

B. Output Program



```
>> Binary Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Binary Search	Best	1000	9	500	1.600
Binary Search	Average	1000	3	374	0.700
Binary Search	Worst	1000	10	Not Found	0.700
Binary Search	Best	10000	13	5000	0.700
Binary Search	Average	10000	13	3745	0.700
Binary Search	Worst	10000	14	Not Found	0.700
Binary Search	Best	100000	16	50000	0.900
Binary Search	Average	100000	16	37454	0.800
Binary Search	Worst	100000	17	Not Found	0.900
Binary Search	Best	1000000	19	500000	2.400
Binary Search	Average	1000000	15	374540	2.600
Binary Search	Worst	1000000	20	Not Found	0.800

Done.
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\searching> |

Gambar 2. Screenshot Hasil Binary Search

C. Pembahasan

1. Karakteristik Algoritma

Secara definisi, Binary Search adalah algoritma pencarian yang bekerja dengan cara membelah ruang pencarian menjadi dua bagian secara berulang-ulang. Karena yang diinginkan adalah membelah ruang pencarian menjadi dua, maka pertama ambil dan tentukan dulu batas `left` dan `right`. Lalu pakai perulangan `while` selama batas `left` lebih kecil atau sama dengan `right` maka akan dilakukan pemotongan rentang. Untuk menentukan sisa rentang mana yang akan dibuang, pertama mengambil nilai tengah (`mid`), kemudian baca nilai pada array di indeks tersebut. Jika nilainya lebih kecil dari target, maka batas `left` digeser ke kanan melewati `mid`. Jika nilainya lebih besar, maka batas `right` digeser ke kiri melewati `mid`. Kalau nilainya ternyata sama persis dengan target, simpan indeksnya lalu `break` untuk keluar dari perulangan.

Mekanisme algoritma ini sangat bergantung pada perbandingan nilai untuk menentukan arah pergeseran rentang, maka algoritma ini memerlukan data terurut. Lalu, karena perulangannya dibuat berputar di dalam blok `while` demi efisiensi memori, maka algoritma ini bersifat iteratif. Meskipun begitu, metode membelah ruang ini dapat diimplementasikan secara rekursif dengan cara memanggil fungsinya sendiri sambil membawa nilai batas `left` atau `right` yang baru terpotong.

2. Kompleksitas Waktu

Secara teori, Best Case Binary Search adalah $O(1)$ karena target seharusnya bisa langsung ditemukan dalam 1 kali tebakan pertama. Namun, pada hasil kode ini, jumlah perbandingan mencatat angka 9 hingga 19 kali operasi akibat posisi target berada di indeks genap (indeks 500) sedangkan perhitungan `int mid = left + (right - left) / 2` menghasilkan pembulatan ke bawah (indeks 499). Karena tebakan awal meleset satu angka, alur program jadi harus melakukan beberapa kali pembelahan ekstra di terminal untuk bergeser menemukan target tersebut (menjadi seperti average case). Meski begitu, nilai kompleksitas teorinya tidak berubah karena jika angka target langsung bertepatan dengan titik tengah pada pengecekan pertama, perintah `if (data[mid] == target)` akan langsung bernilai benar sehingga perintah `break` segera memutus perulangan. Maka, kompleksitas waktu untuk kondisi terbaik dari bagian kode ini secara konsep tetap bernilai $O(1)$.

Namun, jika angka target tidak tertebak pada langkah pertama, program harus membuang setengah bagian data yang tidak sesuai. Algoritma ini didesain untuk mereduksi sisa area pencarian dengan cara menggeser batas `left` atau `right`. Di dalam perulangan `while (left <= right)`, modifikasi batas ini secara konsisten selalu memotong ukuran rentang data menjadi setengah dari ukuran sebelumnya pada setiap putarannya. Karena program tidak menelusuri data satu per satu, melainkan terus membaginya dengan angka dua sampai rentang datanya habis, jumlah operasinya tidak akan bertambah secara drastis meskipun ukuran datanya jutaan. Maka, kompleksitas waktu untuk average dan worst case-nya adalah $O(\log n)$.

3. Analisis Operasi

Berdasarkan hasil pengujian pada terminal kode, jumlah comparison yang dilakukan hingga seluruh data diperiksa terbukti efisien meski tebakan awalnya meleset, pada dataset 1 juta data, Best Case melakukan 19 kali comparison, Average Case 15 kali comparison, dan Worst Case di 20 kali comparison. Jika nilai tengah yang diperiksa cocok dengan target, posisinya langsung disimpan ke variabel `index_found` (indeks 500, 374, atau 500000), namun jika batas rentang data sudah habis terbagi dan target tetap tidak ada, `index_found` akan menampilkan -1 (Not Found).

Karena bagian kode pembelahan rentang ini selalu menebak dari nilai tengah, posisi elemen target tetap berpengaruh terhadap naik-turunnya jumlah comparison yang terjadi. Walau begitu, karena alur programnya selalu membuang setengah area data pada setiap putaran `while`, maka membesarnya ukuran data (n) dari 1.000 ke 1.000.000 elemen hanya memberikan pengaruh penambahan jumlah comparison yang sangat kecil.

4. Analisis Performa

Pada dataset kecil ($N = 1.000$), waktu eksekusi Linear dan Binary Search sama-sama efisien karena berada di bawah $10 \mu s$. Namun pada dataset besar ($N = 1.000.000$), waktu Linear Search naik drastis hingga $11.425 \mu s$, sedangkan Binary Search stabil di angka $0,8 - 2,6 \mu s$. Karena Binary Search mampu menahan kenaikan waktu pada data masif, maka Binary Search jauh lebih efisien untuk dataset besar, sementara Linear Search hanya efisien untuk dataset kecil.

Dikarenakan Binary Search bekerja dengan metode membelah rentang angka, algoritmanya akan gagal beroperasi jika data yang diberikan acak. Sebaliknya, Linear Search mencari dengan cara menelusuri satu per satu data sehingga tidak terpengaruh oleh susunan datanya. Maka, kondisi data yang terurut adalah syarat wajib bagi performa Binary Search, namun tidak memberikan pengaruh sama sekali pada Linear Search.

Pada pengujian 1 juta data (worst case), Linear Search tercatat melakukan 1.000.000 kali comparisons yang menyebabkan lonjakan waktu linear. Sebaliknya, Binary Search hanya membutuhkan maksimal 20 kali comparisons. Karena angka maksimal 20 pada pembelahan data ini sejalan dengan hitungan $\log_2(1000000) \approx 19.93$, maka hasil ini membuktikan kebenaran teori kompleksitas $O(n)$ untuk Linear Search dan $O(\log n)$ untuk Binary Search.

TAUTAN GITHUB

<https://github.com/DSA26-ULM/module-5-searching-nullrym>